

Part 1: the perceptron

(1) 感知机的学习过程理论分析：

感知机模型：感知机是一种线性分类器，用来处理两类线性可分的数据。它根据输入特征的加权和来预测类别，如果加权和大于等于 0，则输出 1，否则输出-1。

学习规则：感知机的学习过程基于感知机学习规则，主要目的是通过调整权重使模型能够正确分类数据。每次训练时，感知机计算当前权重下的预测结果，然后通过与实际标签的差异（误差）来调整权重。直到模型收敛或达到最大迭代次数。

批量梯度下降（Batch Gradient Descent）：在本任务中，我们采用批量梯度下降，每个训练周期（epoch）中，模型遍历所有数据点并在遍历完成后一次性更新权重。这种方法收敛相对稳定，但由于需要处理整个数据集，计算开销较大。

(2) 任务实现思路

1、使用 numpy 中的 **函数名称** 函数从每个高斯分布中采样 100 个点。
将生成的数据分为训练集和测试集。

2. 初始化感知机：我们要初始化感知机的参数，包括输入的维度、最大训练周期（epochs）和学习率（learning rate），以及初始化感知机的权重（此处默认设为 0）。

前向传播（forward pass）：通过将输入向量与权重相乘，再加上偏置项（默认为 0），来计算预测的结果（+1 或 -1）。

代码图片

训练过程：根据训练数据进行多次迭代（限定最大次数，若 error 为 0 则提前退出）。在每次迭代中，我们计算每个点的预测值与真实标签的误差，并使用误差更新权重（weight 和 bias 分开处理）。这里使用的是批量梯度下降（batch gradient descent），也就是在处理完整个数据集之后再更新一次权重。

代码图片



3. 完成感知机的实现后，我们使用 160 个训练点对模型进行训练（2 个特征向量）。训练完成后，我们将感知机应用于剩下的 40 个测试点，并计算它的分类准确率。

4. 调整生成的两个高斯分布的均值和方差来进行实验。如果两个分布的均值靠得太近，或者它们的方差太大，训练过程会变得更加困难。这是因为数据点之间的重叠增多，导致分类边界难以划分。通过观察不同的实验结果，可以了解感知机在处理有高度重叠的分布时的局限性（比如需要高维曲线作为边界）。

Testcase:

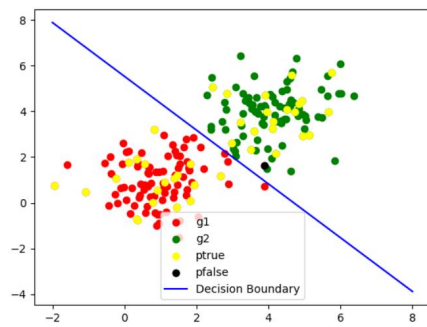
Casel: common case

测试样例



样例结果

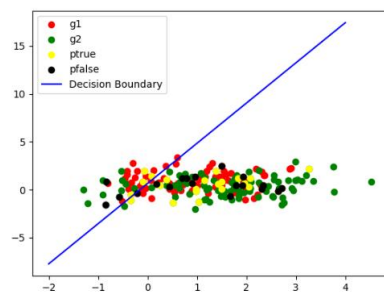




Case2: similar mean value

测试样例

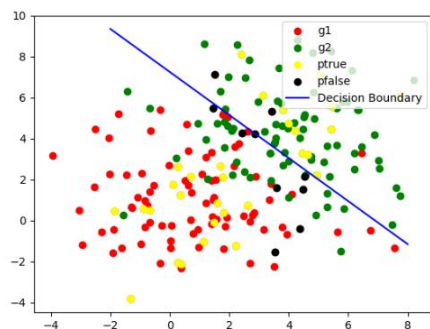
样例结果



Case3: large cov

测试样例

样例结果



2 Part II: the mutli-layer perceptron

(1) 多维感知机的学习过程理论分析：

MLP

数据输入：首先，输入层接收原始数据，此处为特征向量。

前向传播：数据从输入层经过隐藏层逐层传递到输出层。每一层的神经元会根据输入计算输出（前向传播）。每个神经元会加权输入并通过激活函数（如 ReLU 或 Sigmoid）进行处理，以增加网络的非线性能力。

输出生成：最后，输出层生成预测结果，比如分类概率。

反向传播：

误差计算：首先，根据网络输出和真实标签计算损失，这个损失代表了模型的预测与实际结果之间的差距。

梯度传递：反向传播从输出层开始，将损失反向传递到每一层。每一层根据当前层的输出和前一层的输入来计算自己的梯度。

更新参数：得到梯度后，使用这些信息来调整每一层的权重和偏置，目的是减少下一次训练时的损失。具体来说，梯度指示了如何调整参数以便更好地匹配数据。

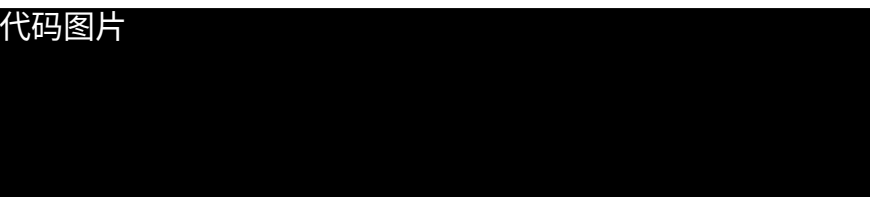
逐层处理：这个过程是逐层进行的，从最后一层到第一层，每层都更新自己的权重和偏置，并将梯度传递给前一层。

难点：

1. 对于矩阵链式求导不熟悉，查阅了较多资料后才大概搞明白。
2. 开始训练时出现了梯度爆炸现象，后来调整了学习率，然后修改了下 *CrossEntropy*

的前向传播函数

代码图片

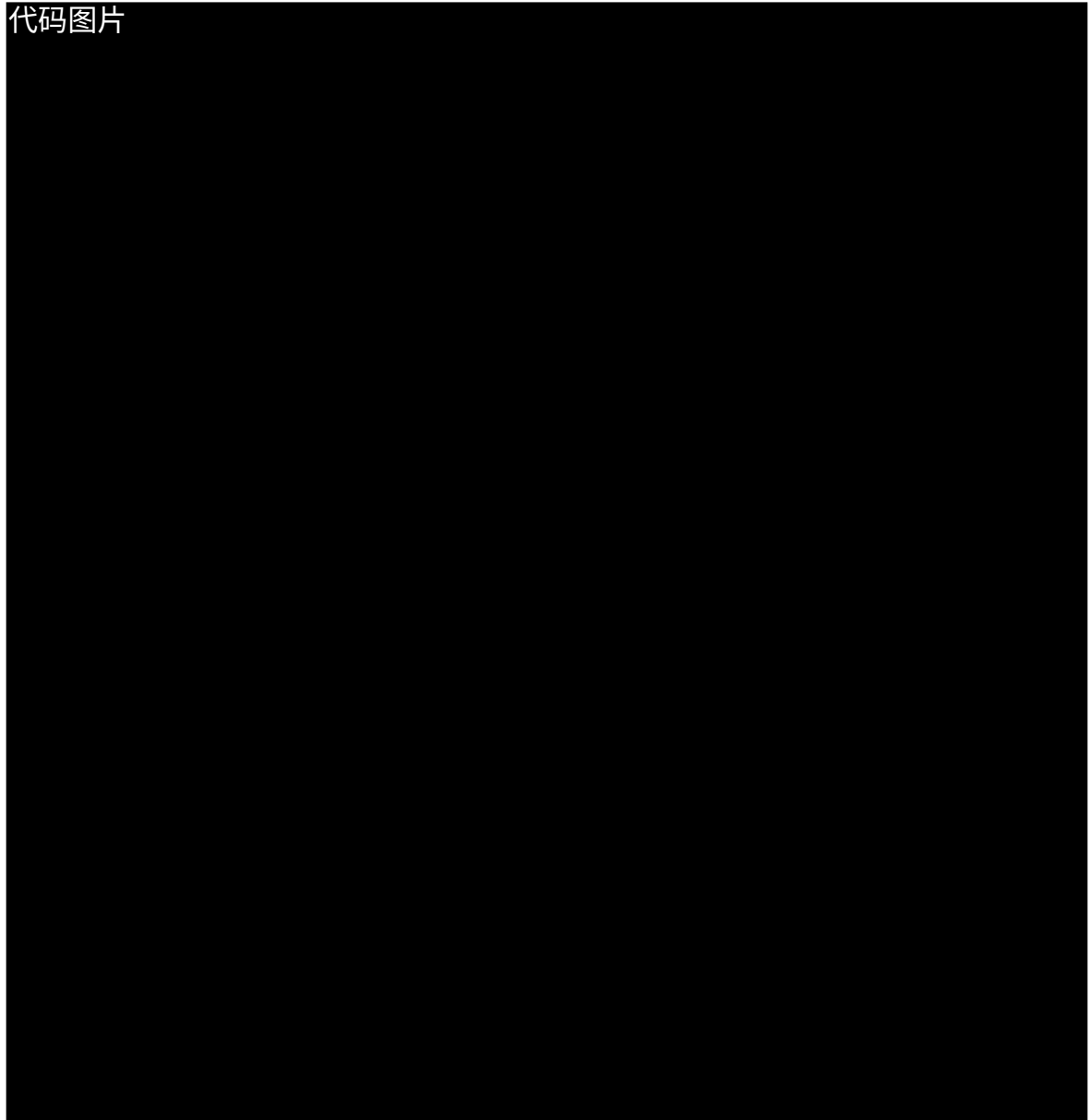


3. 加入了  减少了计算量。

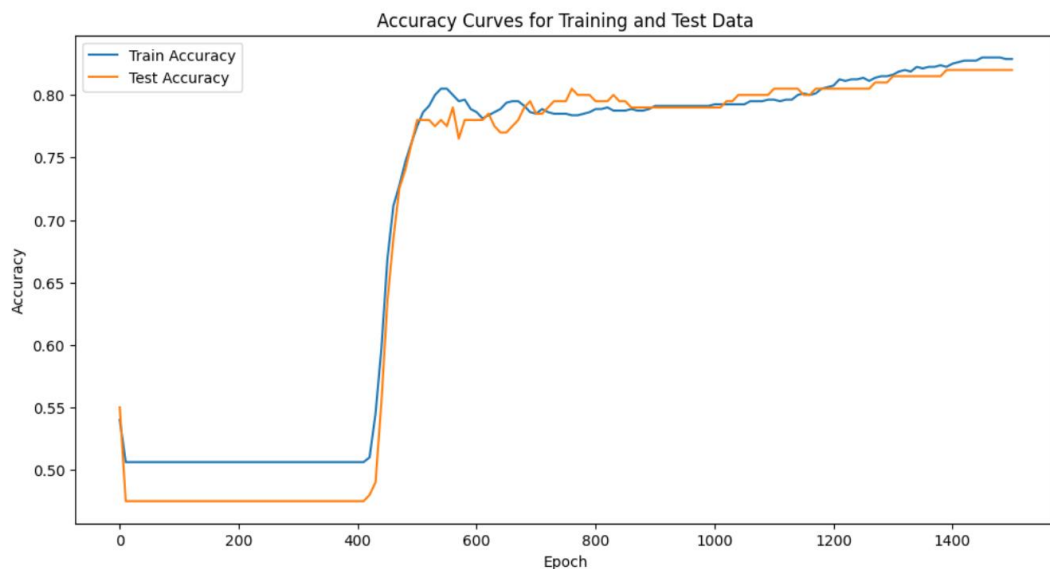
(2) 任务实现思路

按照模块，分层一步步处理多层感知机。并为各层采用合适的激活函数。通过正向传播得出结果，同时，通过反向传播逐层调整权重。（代码里给的 todo 提示很详细，所以不再赘述了）

代码图片



(3) 测试结果：



从输出结果来看，模型在训练过程中逐渐收敛。我们从以下几个方面分析：

1. 训练准确率从最初的约 75% 提高到了最终的约 83%
2. 测试损失也随着训练步骤的增加而逐渐下降，但仍较高，待改进（图中未显示 loss，文字描述中 loss 从 0.7 降至 0.4）。
3. 过拟合情况：从训练准确率和测试准确率的变化趋势来看，两者之间的差距并不大，说明模型没有出现严重的过拟合现象。
4. 收敛情况：最后几个 epoch 中，训练损失和测试损失的下降速度变慢，欠拟合现象不明显。

Part III: stochastic gradient descent

（1）理论分析：

随机梯度下降：

1. 逐样本更新：在 SGD 中，每次迭代只使用一个训练样本来计算梯度。这意味着每次更新模型参数时，都会考虑最新的数据点。这种逐样本的更新方式使得参数更新更加频繁，从而可以更快地调整模型。
2. 减少计算量：由于每次只处理一个样本，SGD 的计算开销相对较小，尤其是在处理大规模数据集时。传统的批量梯度下降需要遍历整个数据集，这在数据量庞大时会非常耗时。
3. 更强的随机性：SGD 的随机性使得模型参数更新过程中的“噪声”更大，这有助于跳出局部最优解，找到全局最优解。虽然每次更新可能不是在下降的最优路径上，但随机性可以帮助模型探索更广泛的解空间。

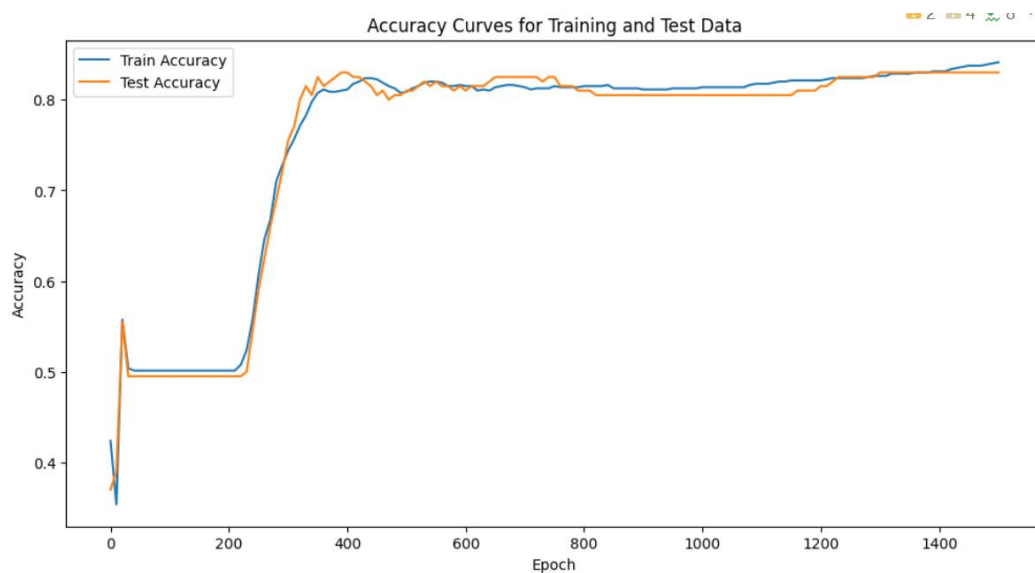
的前向传播函数

（2）实现思路

每次只处理一个样本，（当然可以考虑让 batchsize 变大些）

代码图片

(3) 测试结果：



Training complete!

从输出结果来看：

1. 训练准确率提升到 85%。
2. 训练过程中有上升的路径，但最后找到了全局最优。